



8. Linked Lists

Connecting Data One Node at a Time

Previous Section:

 [7. Arrays](#)

Contents:

[1. What is a Linked List?](#)

[2. Types of Linked Lists](#)

[2.1. Singly Linked List](#)

[2.2. Doubly Linked List](#)

[2.3. Circular Linked List](#)

[2.4. Header Linked List](#)

[3. Applications of Linked Lists](#)

[**Advantages of Linked Lists**](#)

[Disadvantages of Linked Lists](#)

[Summary](#)

[References](#)

In the previous section, we explored arrays, one of the most commonly used data structures in programming. Arrays are simple, efficient, and provide fast access to elements through indexing. However, they also come with several limitations. In many programming languages, arrays have a fixed size, meaning we must determine their capacity when they are created. If we need more space later, resizing the array can be expensive.

Another challenge appears when inserting or deleting elements. Since array elements are stored next to each other in memory, adding or removing an

element often requires shifting many other elements to maintain the correct order. As the size of the array grows, these operations become increasingly costly.

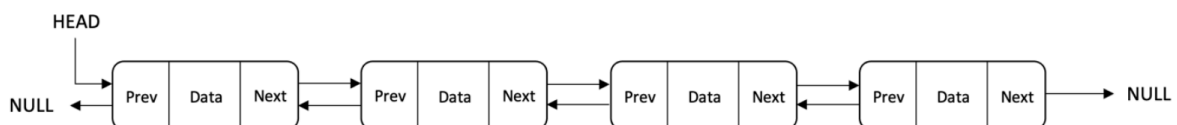
To overcome these limitations, we can use a different type of linear data structure known as a **Linked List**. Unlike arrays, linked lists store data in separate nodes that are connected together through references. This design allows the structure to grow dynamically and makes insertion and deletion operations much more efficient in many situations.

In this section, we will learn how linked lists work, how they differ from arrays, and why they are an important building block for many advanced data structures and algorithms.

1. What is a Linked List?

A **Linked List** is a linear data structure made up of a collection of nodes connected together through references (often called pointers). Each node contains two main components:

- A data field that stores the actual value.
- A reference to the next node in the sequence.



Unlike an array, the nodes of a linked list do not need to be stored in consecutive memory locations. Each node can exist anywhere in memory, and the references are used to connect them together in the correct order. Because of this, linked lists can grow and shrink dynamically as needed.

We can think of a linked list as a chain. Each link in the chain represents a node, and each node knows where the next node is located. By starting from the first node and following these references, we can traverse the entire list from beginning to end.

One of the greatest advantages of linked lists is their flexibility. Since nodes can be inserted or removed by simply adjusting references, many insertion and deletion operations can be performed without shifting large amounts of data. This makes linked lists particularly useful when the size of the collection changes frequently.

To better understand their characteristics, let us compare linked lists with arrays:

- **Arrays** store elements in contiguous memory locations, allowing direct access to any element using its index. This makes element access extremely fast.
- **Linked Lists** store nodes in non-contiguous memory locations. Accessing a specific element requires traversing the list from the beginning until the desired node is reached.
- **Arrays** require shifting elements during many insertion and deletion operations.
- **Linked Lists** can often perform insertion and deletion more efficiently by updating references between nodes.
- **Arrays** generally use less memory because they store only the data.
- **Linked Lists** require additional memory to store references that connect the nodes together.

As a result, arrays are often preferred when fast random access is required, while linked lists are valuable when flexibility and frequent modifications to the collection are more important.

2. Types of Linked Lists

Linked lists come in several different forms. Although all linked lists are built from nodes connected together through references, the way these nodes are connected can greatly affect how the list behaves and what operations can be performed efficiently.

Some linked lists allow movement in only one direction, while others allow movement in both directions. Some end with a special NULL reference, while others form a continuous loop. There are even linked lists that contain special header nodes used to store additional information about the structure itself.

Understanding these variations helps us choose the most suitable linked list for a particular problem.

2.1. Singly Linked List

A **Singly Linked List** (also known as a **One-Way Linked List**) is the simplest form of linked list. Each node stores a piece of data and a reference to the next node in the sequence.

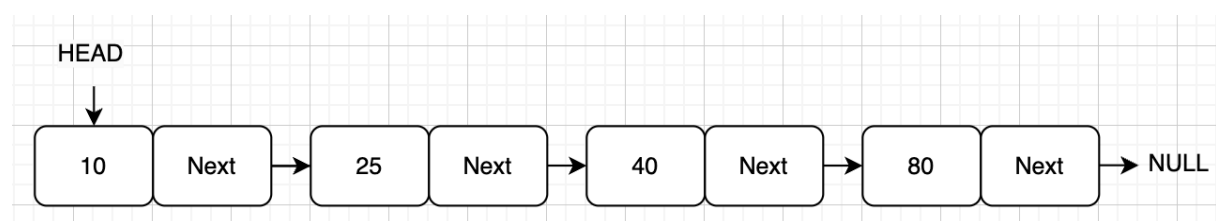
Every node contains two components:

- **Data (Value)** – stores the information held by the node.
- **Next** – stores a reference to the next node in the list.

The first node of the list is called the **Head** (or **First** node). The final node points to **NULL**, indicating that there are no more nodes after it.

A singly linked list can only be traversed in one direction, from the head toward the tail.

Example:



In this example: The head node contains 10. Each node points to the next node. The final node contains 80 and points to NULL.

Because each node only knows where the next node is located, moving backward through the list is not possible.

2.2. Doubly Linked List

A **Doubly Linked List** (or **Two-Way Linked List**) extends the idea of a singly linked list by storing two references inside every node.

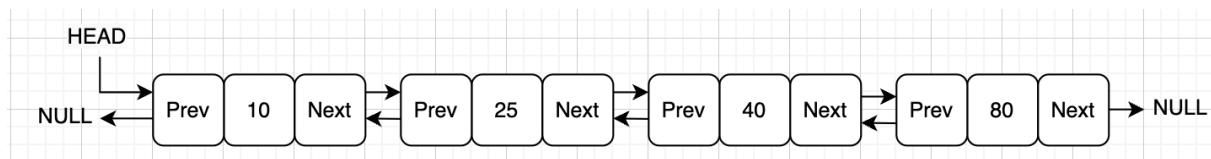
Each node contains:

- **Data (Value)**

- **Prev** – points to the previous node.
- **Next** – points to the next node.

Since every node knows both its predecessor and successor, we can traverse the list in either direction.

Example:



This structure allows efficient insertion and deletion because neighboring nodes can easily update their references.

Compared to singly linked lists, doubly linked lists require more memory because every node stores an additional pointer.

2.3. Circular Linked List

A **Circular Linked List** is a linked list in which the final node does not point to NULL. Instead, it points back to the beginning of the list, creating a circular structure.

Because there is no NULL reference at the end, traversal can continue indefinitely unless a stopping condition is used.

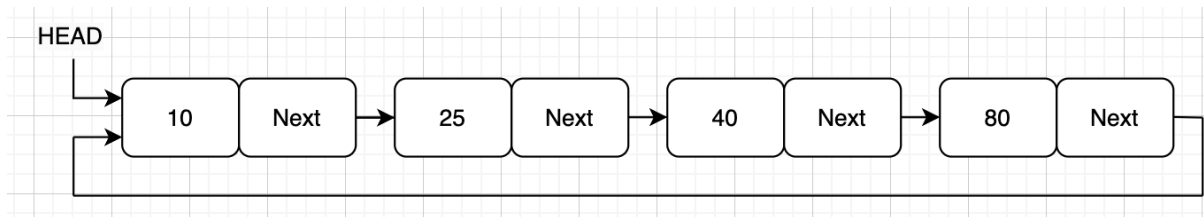
Circular linked lists are often used in scheduling systems, playlists, and applications that repeatedly cycle through a collection of items.

There are two major types of circular linked lists.

Circular Singly Linked List

A **Circular Singly Linked List** is similar to a regular singly linked list, except the last node points back to the first node.

Example:



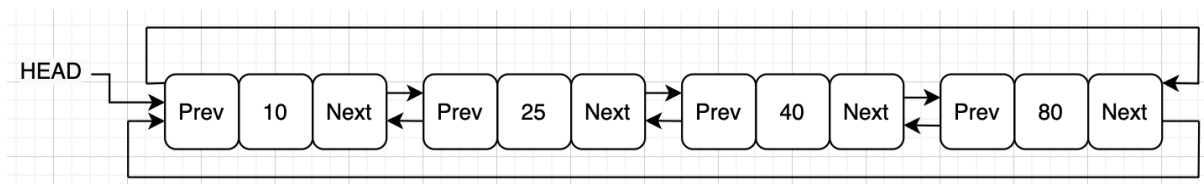
Notice that:

- There is no NULL reference.
- The final node links back to the head node.
- Traversal can continue forever in one direction.

Circular Doubly Linked List

A **Circular Doubly Linked List** combines the features of a doubly linked list and a circular linked list. Each node stores both Prev and Next references, and the first and last nodes are connected together.

Example:



In this structure:

- The Next pointer of the last node points to the first node.
- The Prev pointer of the first node points to the last node.
- Traversal is possible in both directions indefinitely.

2.4. Header Linked List

A **Header Linked List** is a special type of linked list that begins with an additional node known as the **Header Node**. Unlike normal nodes, the header node usually stores information about the list itself rather than user data.

Common information stored in a header node includes:

- Number of nodes in the list
- Current size of the list
- References to important nodes
- Other metadata required by an application

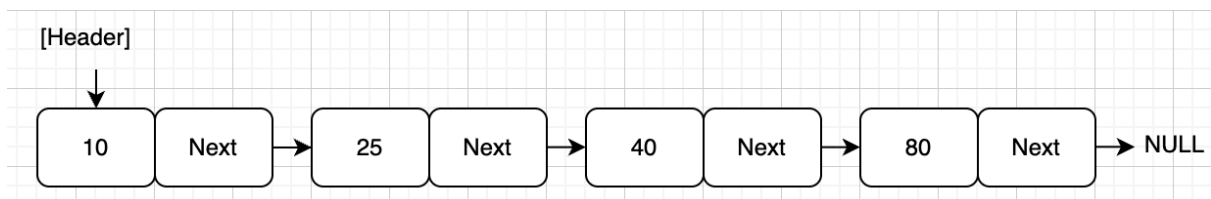
The presence of a header node can simplify many linked list operations.

For example, if we frequently need to know the number of nodes in a list, storing that value inside the header node avoids traversing the entire list every time.

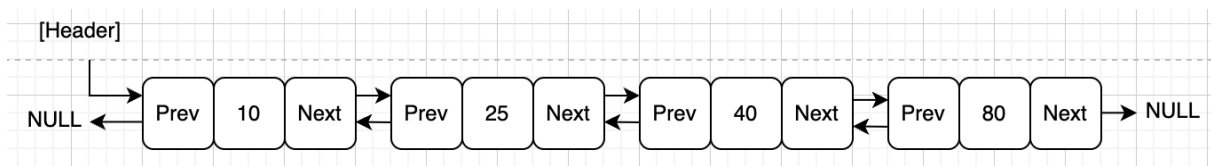
Singly or Doubly Header Linked List

These structures behave similarly to ordinary singly or doubly linked lists, except that they begin with a header node.

- **Singly Header Linked List**



- **Doubly Header Linked List**

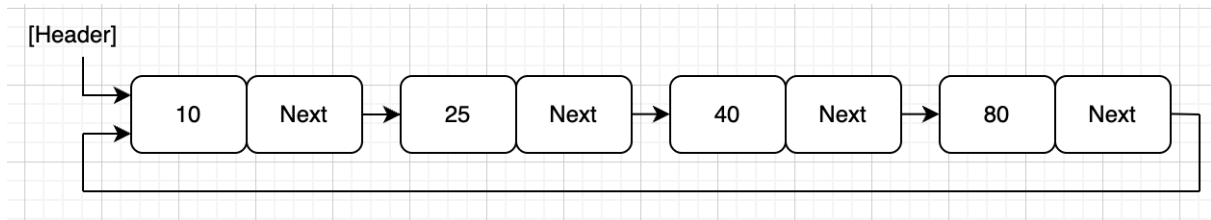


The header node acts as a placeholder and often simplifies insertion and deletion operations near the beginning of the list.

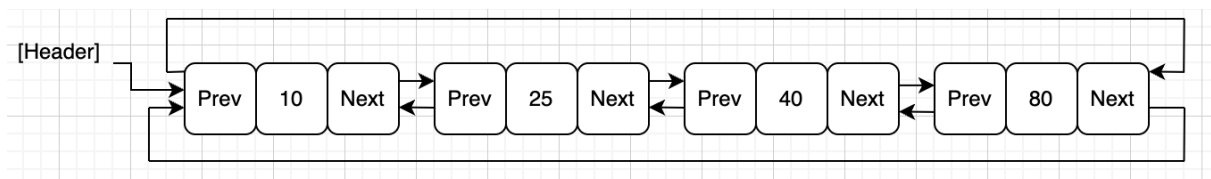
Singly or Doubly Circular Header Linked List

A **Circular Header Linked List** combines a header node with a circular structure. Instead of pointing to NULL, the last node points back to the header node.

- **Singly Circular Header Linked List**



- **Doubly Circular Header Linked List**



The header node serves as a permanent reference point, making traversal and manipulation easier because there is no need to handle special "first" or "last" node cases.

3. Applications of Linked Lists

Linked lists are used extensively throughout computer science because of their flexibility and dynamic nature.

Some common applications include:

- **Implementing Other Data Structures:** Many important data structures are built using linked lists, including Stacks; Queues; Deques; Sparse Matrices; Graph Adjacency Lists. The ability to insert and remove nodes efficiently makes linked lists a natural choice for these structures.
- **Operating Systems:** Operating systems frequently use linked lists for dynamic memory allocation, memory management, process scheduling, file system management. For example, Round Robin Scheduling often cycles through processes in a manner similar to a circular linked list.

- **Music Playlists:** A playlist in a music application behaves much like a linked list.

For example: `Song A ⇌ Song B ⇌ Song C ⇌ Song D`

The previous and next buttons allow movement through the songs in either direction.

- **Web Browsing History:** Most web browsers maintain navigation history using a structure similar to a doubly linked list.

For example: `Page 1 ⇌ Page 2 ⇌ Page 3 ⇌ Page 4`

The Back button moves to the previous page, while the Forward button moves to the next page.

- **Image Viewing Systems:** Image viewers often connect images in a sequence.

For example: `Image 1 ⇌ Image 2 ⇌ Image 3 ⇌ Image 4`

This allows users to navigate between images using Previous and Next controls.

Advantages of Linked Lists

Linked lists provide several benefits compared to arrays.

- **Efficient Insertion and Deletion:** Inserting or deleting a node usually requires updating only a few references. Unlike arrays, there is no need to shift large groups of elements.
- **Dynamic Size:** Linked lists can grow or shrink as needed. We do not need to determine the size of the structure in advance.
- **Easy Implementation of Dynamic Data Structures:** Many dynamic data structures are naturally implemented using linked lists because nodes can be added or removed efficiently.
- **Better Memory Usage in Unpredictable Situations:** When the number of elements is unknown, linked lists can be more memory-efficient because memory is allocated only when needed.

Disadvantages of Linked Lists

Despite their flexibility, linked lists also have several drawbacks.

- **No Direct Index Access:** To access an element, we must traverse the list from the beginning. This results in a time complexity of $O(n)$ for searching.
- **More Complex Structure:** The use of references and node management makes linked lists more difficult to understand, implement, and debug than arrays.
- **Additional Memory Overhead:** Each node requires extra memory to store one or more references. This overhead can become significant in large linked lists.
- **Cache Inefficiency:** Nodes are often scattered throughout memory. Because the data is not stored contiguously, modern processors cannot take advantage of caching as effectively as they can with arrays. This can lead to slower traversal performance.
- **No Built-in Support in Many Languages:** Although many libraries provide linked list implementations, understanding and implementing linked lists often requires manual management of nodes and references. This can be challenging for beginners.

Summary

In this section, we explored **Linked Lists**, one of the most important dynamic data structures in computer science. Unlike arrays, which store elements in contiguous memory locations, linked lists store data inside nodes that are connected through references. This design allows linked lists to grow and shrink dynamically, making insertion and deletion operations much more efficient in many situations.

We examined the major types of linked lists, including **Singly Linked Lists**, **Doubly Linked Lists**, **Circular Linked Lists**, and **Header Linked Lists**. Although each type organizes nodes differently, they all share the same fundamental idea: storing data in nodes that are linked together rather than placed side by side in memory.

We also discussed many real-world applications of linked lists, from operating systems and memory management to music playlists, web browser navigation, image viewers, and the implementation of other data structures such as stacks,

queues, and graphs. These applications demonstrate why linked lists remain a fundamental building block in software development.

Like every data structure, linked lists come with both strengths and weaknesses. Their dynamic size and efficient insertion and deletion operations make them highly flexible, but they also require additional memory for references and do not provide the fast random access that arrays offer.

At this point, we have a general understanding of how linked lists work and why they are useful. However, most linked list implementations are built upon two foundational structures: the **Singly Linked List** and the **Doubly Linked List**. These two forms serve as the basis for many of the more advanced variations we have discussed.

In the next section, we will move from theory to implementation by exploring how **Singly Linked Lists** and **Doubly Linked Lists** are constructed, traversed, modified, and used in practical programming scenarios.

References

https://drive.google.com/file/d/1xW6GD_okYUEMMlotBxcIA5RdAe2Rx8Gu/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/linked-list-data-structure/>

Next Sections:

 [8.1. Singly Linked Lists](#)

 [8.2. Doubly Linked Lists](#)

 [8.3. Special Operations on Linked Lists](#)